AutoLISP Runtime

Specifications

clautolisp contributors

May 25, 2026

Contents

1 Introduction

The `clautolisp.autolisp-runtime` system implements the AutoLISP evaluator and the supporting environment model: symbol table, binding cells,

dynamic-frame stack, namespaces, dialect descriptors, runtime sessions, and evaluation contexts. It is the core layer the upper subsystems (builtins, HAL, DCL, file-compat) attach to.

Audience: clautolisp developers extending the evaluator or implementing new builtin categories.

2 Goals

- Implement AutoLISP semantics faithfully on top of Common Lisp, preserving the dynamic-scope rule, the Lisp-1 single-binding-cell contract, and the autolisp-spec normative behaviour.
- Keep AutoLISP runtime values distinguishable from Common Lisp values (`autolisp-string`, `autolisp-symbol`, `autolisp-subr`, `autolisp-usubr`, `autolisp-ename`, …) so the printer and host bridge surface them with the right syntax.
- Stay portable across SBCL and CCL.

3 The single binding cell (Lisp-1)

AutoLISP is a Lisp-1: SETQ and DEFUN write to the same per-symbol binding cell. The runtime models this with `binding-cell` in `source/model.lisp`:

(defstruct binding-cell (value nil) (bound-p nil :type boolean) (compatibility-definition nil :type list))

The optional `compatibility-definition` slot is the side channel DEFUN-Q populates with a list-form copy of the function; plain SETQ and DEFUN clear it. See autolisp-spec ch. 7 "Single-Cell Symbol Binding (Lisp-1)" for the normative rule and its vendor evidence.

4 Dynamic scope

Local bindings introduced by `/` arglist tails, lambda parameters, and FOREACH iterator variables push entries onto a `dynamic-frame` stack threaded through the `evaluation-context`. `lookup-variable` walks the frame chain first, then falls back to the namespace's `binding-cell`. `set-variable` mirrors the lookup shape: it updates the innermost dynamic shadow when one exists, otherwise the namespace cell.

The same dual-path discipline applies to `lookup-function` and `set-function` — preserving the Lisp-1 rule that a value-position shadow is also a function-position shadow (and vice versa).

5 Special operators

`*special-operator-dispatch*` maps an uppercase operator name (QUOTE, SETQ, PROGN, IF, COND, AND, OR, WHILE, REPEAT, FOREACH, LAMBDA, FUNCTION, DEFUN, DEFUN-Q) to a one-shot evaluator function. Operator names are NOT interned in the symbol table — the evaluator recognises them by string equality of the operator's symbol-name before normal apply dispatch.

6 Sessions and contexts

A `runtime-session` owns a document namespace and a global blackboard, plus the dialect descriptor and an optional HAL backend. An `evaluation-context` refers back to its session and carries the current dynamic-frame chain — the value the evaluator threads through every internal call.

The session-level `default-source-encoding` slot is the lever the CLI's `-e ENC` option flips: when set, every `load` within the session honours it instead of the dialect default.

7 Dialect descriptors

A dialect descriptor (from `clautolisp.autolisp-reader`) carries the reader-level knobs: token mode, integer-overflow handling, extended-string-escape behaviour, canonical case, unbound-variable mode, default source encoding. The runtime queries the descriptor when evaluating unbound symbols, choosing between `:silent-nil` (AutoCAD / BricsCAD profiles) and `:strict-error` (a non-conforming diagnostic mode useful in unit tests).

8 Reader-to-runtime conversion

`reader-object->runtime-value` and `reader-objects->runtime-values` map the structured AST the reader produces into the Common Lisp shape the evaluator consumes:

- `symbol-object` → `autolisp-symbol`

- `string-object` → `autolisp-string`
- `cons-object` → plain CL list (mapcar over elements)
- `quote-object` → `(QUOTE …)` list

This is a one-way handoff — the runtime does NOT preserve the reader's source-span metadata at this layer (the source-aware- defun-documentation issue plans an EQ-hash side table for forms that need preceding-doc carry-through).

9 Test coverage

Tests live under `tests/` and cover the runtime in isolation (without the builtins layer). Key suites:

- `evaluator-tests.lisp` — end-to-end run-autolisp-string + Lisp-1 contract + unbound-variable dialect behaviour + T self-evaluation.
- `namespace-tests.lisp` — namespace-binding-cell + propagation
  - document-namespace lifecycle.
- `dynamic-frame-tests.lisp` — push/pop, find-dynamic-binding, shadow semantics.

The runtime is intentionally tested without the builtins layer loaded; tests install a small set of test-only subrs (LIST, ADD2) via `install-list-subr-for-test` when a snippet needs one.

10 Open questions

- Source-aware documentation (`issues/open/source-aware-defun-documentation.issue`): how to carry `:preceding-doc` from the reader's cons-object onto the CL cons the runtime sees, then surface it via `(clautolisp-documentation 'FOO)`. The design choice between EQ-hash side-table vs annotated cons-object lives there.
- Performance: the dynamic-frame walk is currently O(n) on chain depth. A frame-local hash could accelerate it for deeply-nested call stacks, but profile data hasn't shown it as a hot spot.

11 References

- Source: `source/api.lisp`, `source/model.lisp`, `source/ontology.lisp`.
- Tests: `tests/`.
- Sibling track: `autolisp-runtime-user-manual.org`.
- Spec: `autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org`.

12 Synchronisation

Per AGENTS.md (Documentation Synchronisation), changes to the code, this spec, or the user-manual must be propagated to the other tracks. This subproject is a library and has no man page.